Step 7: Sum weighted values to get Output 1

Sum the weighted values
To get output for position 1

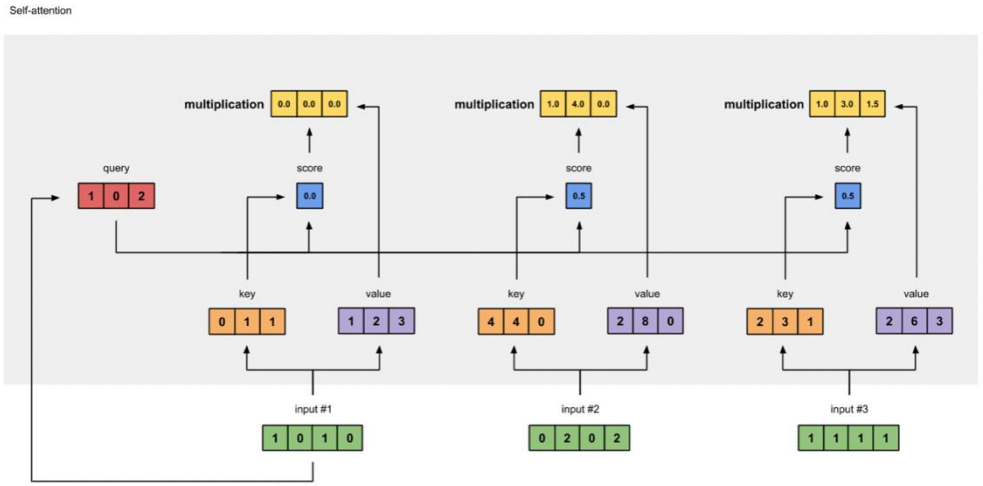


Fig. 1.7: Sum all **weighted values** (yellow) to get Output 1 (dark green)

Take all the **weighted values** (yellow) and sum them element-wise:

```
  [0.0, 0.0, 0.0]
+ [1.0, 4.0, 0.0]
+ [1.0, 3.0, 1.5]
-----------------
= [2.0, 7.0, 1.5]
```

The resulting vector [2.0, 7.0, 1.5] (dark green) is Output 1, which is based on the **query representation from Input 1** interacting with all other keys, including itself.

Step 8: Repeat for Input 2 & Input 3

Now that we're done with Output 1, we repeat Steps 4 to 7 for Output 2 and Output 3. I trust that I can leave you to work out the operations yourself 👍🏼.

Repeat for input 2 and 3

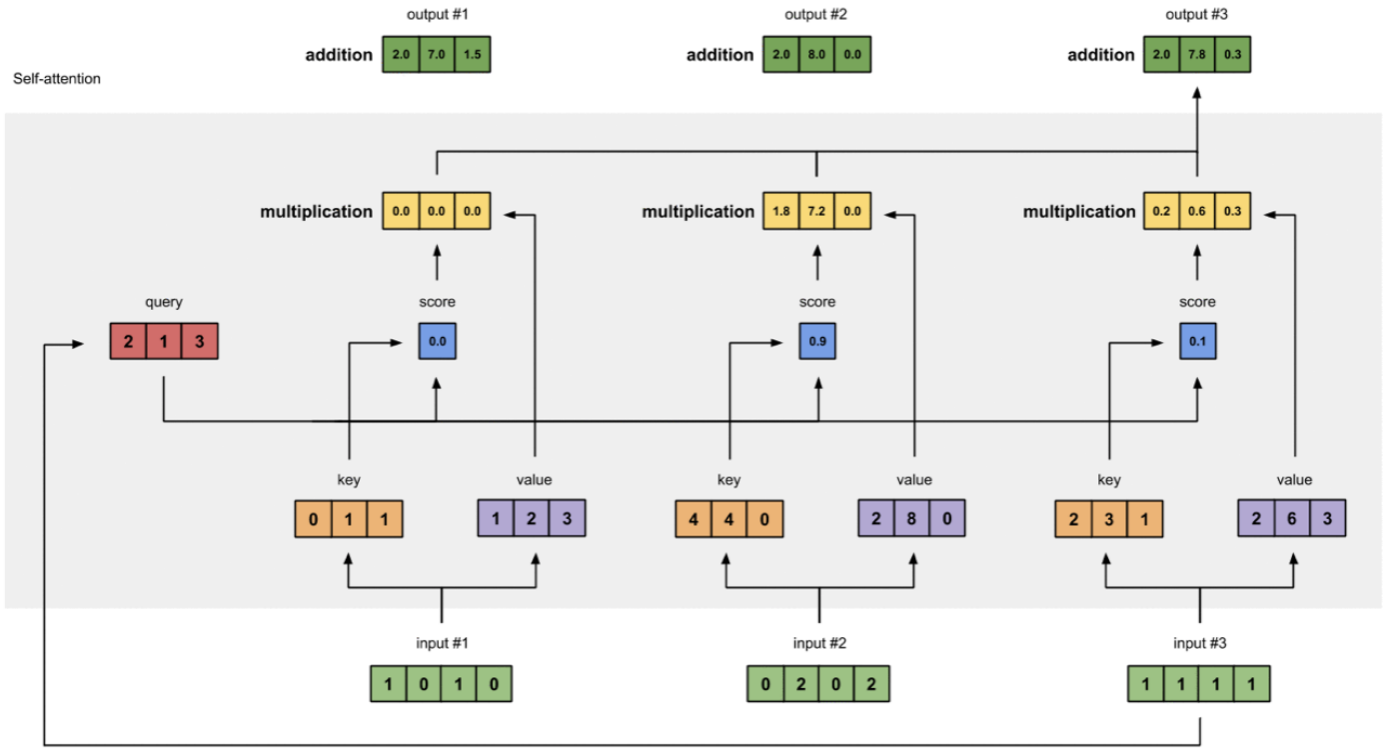
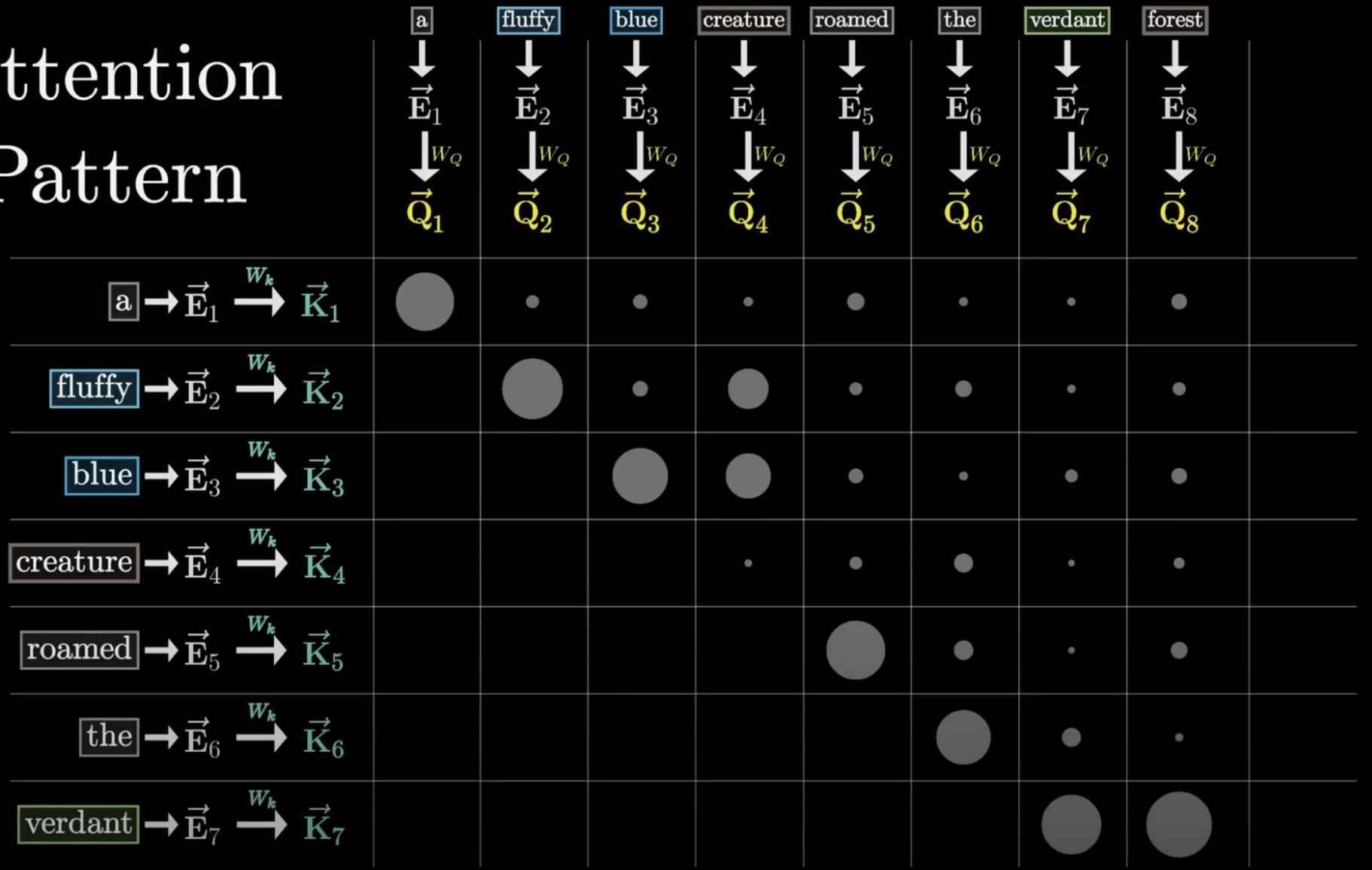
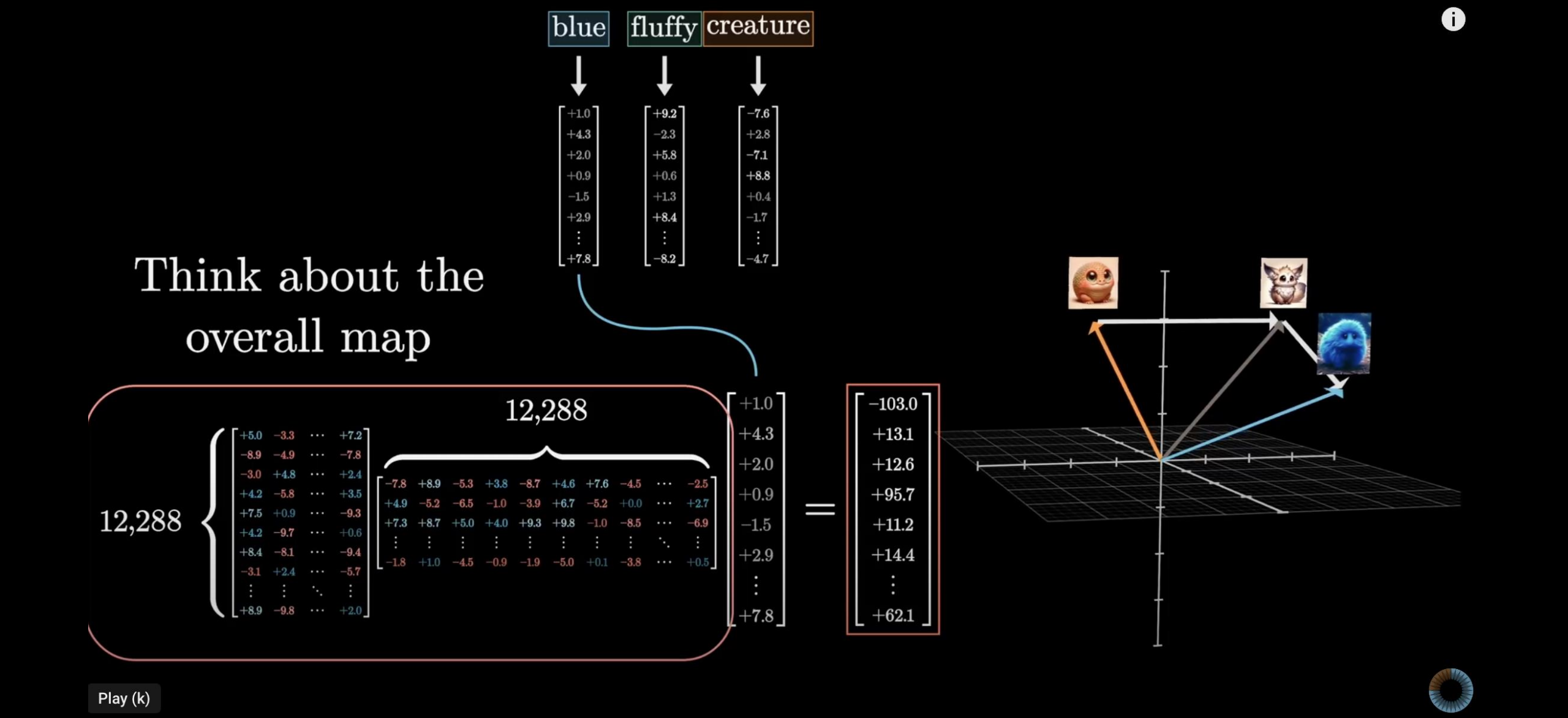


Fig. 1.8: Repeat previous steps for Input 2 & Input 3

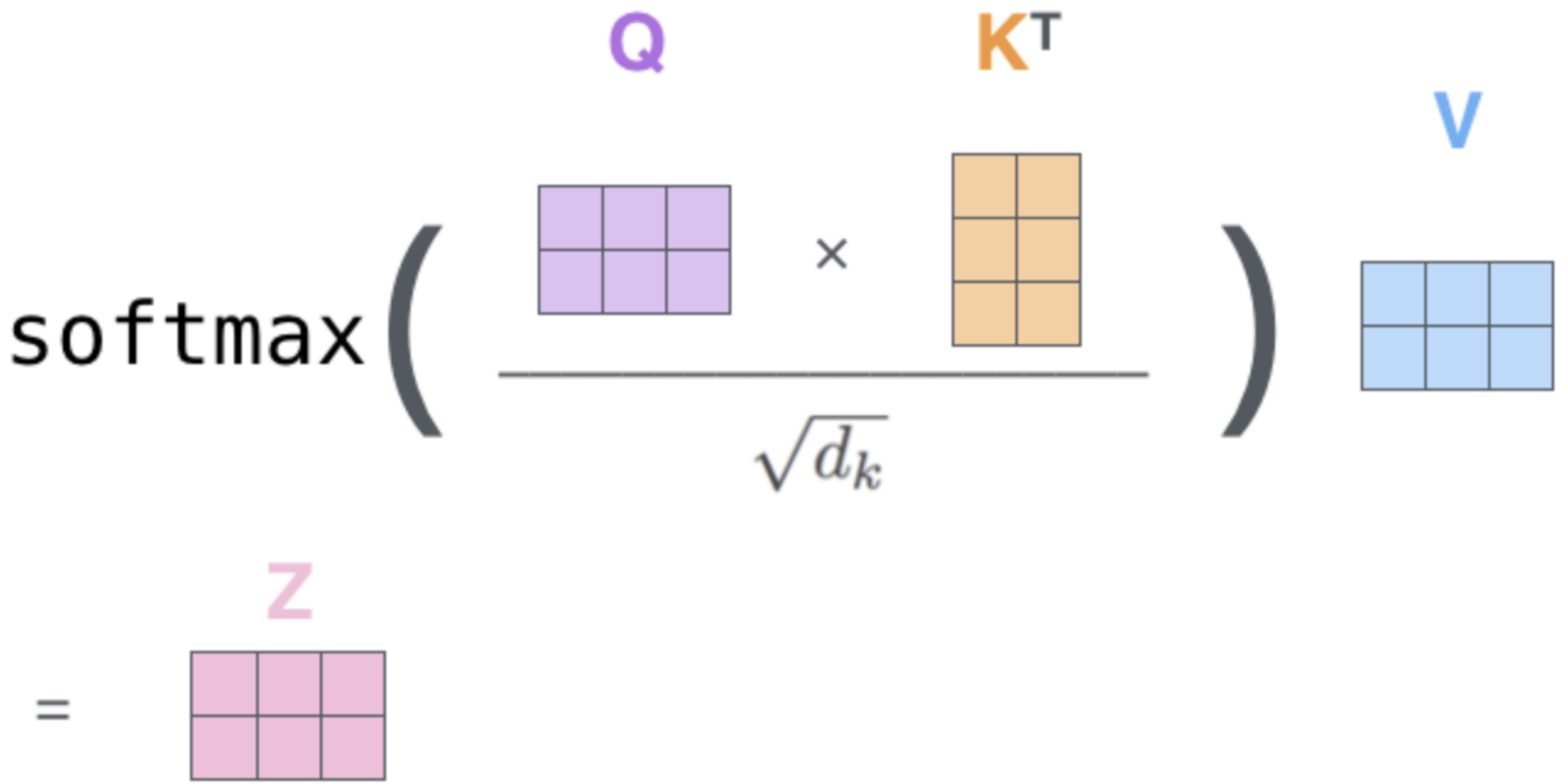
Attention Pattern




blue
fluffy
creature
Think about the overall map
12,288
12,288
Play (k)

> **Notes**
>
> The dimension of **query** and **key** must always be the same because of the dot product score function. However, the dimension of **value** may be different from **query** and **key**. The resulting output will consequently follow the dimension of **value**.

Finally, since we're dealing with matrices, we can condense steps two through six in one formula to calculate the outputs of the self-attention layer.



The self-attention calculation in matrix form

Multihead attention

Multiple representation Subspaces

Think of it as multiple Contexts

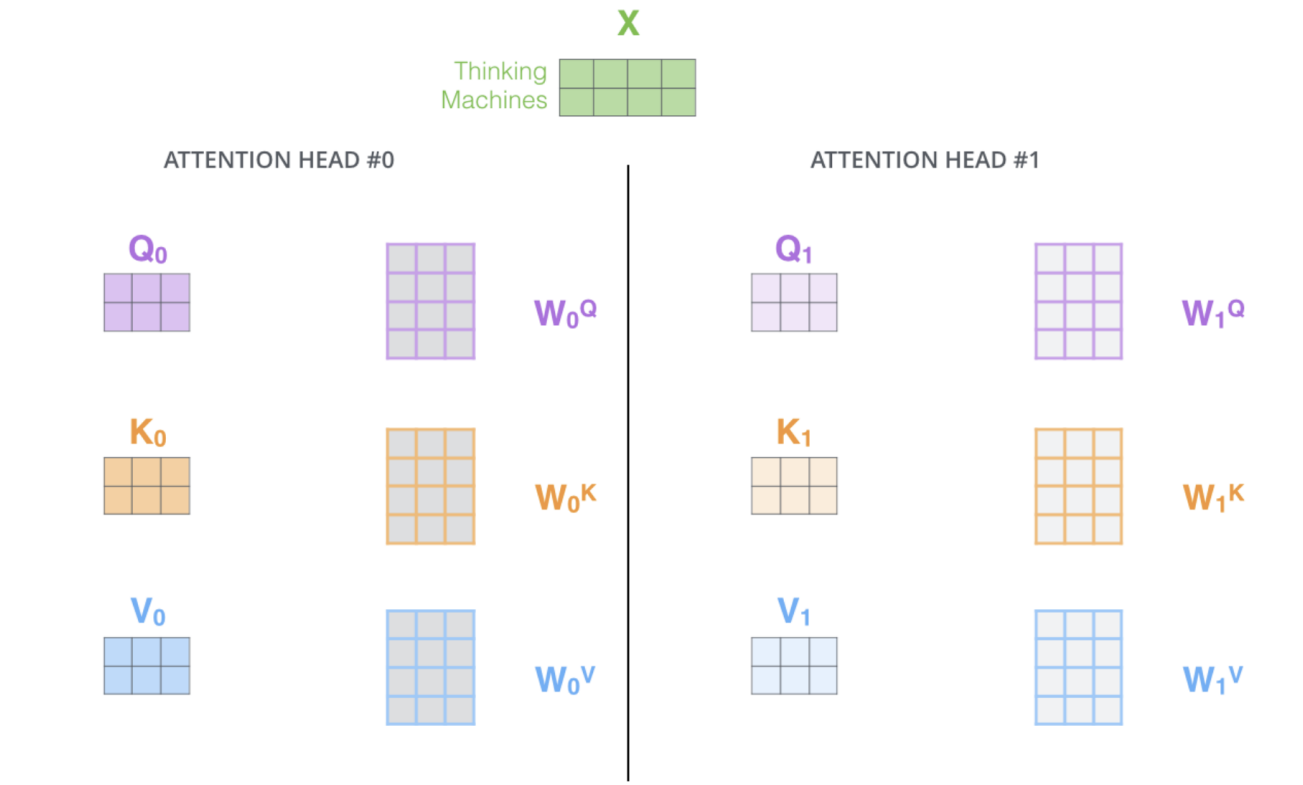
#heads in GPT-3 175B = 96

Same as number of layers

The Beast With Many Heads

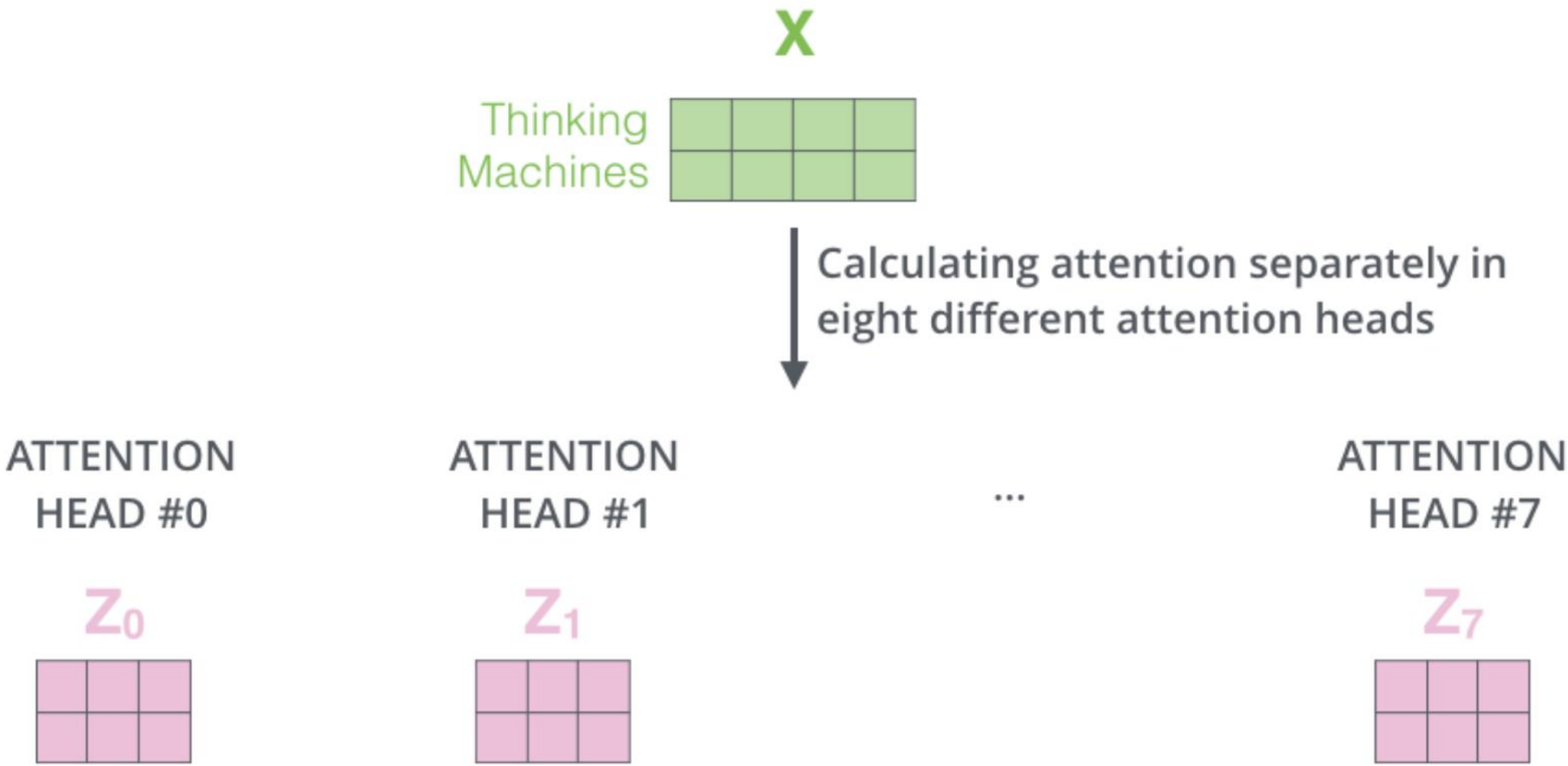
The paper further refined the self-attention layer by adding a mechanism called “multi-headed” attention. This improves the performance of the attention layer in two ways:

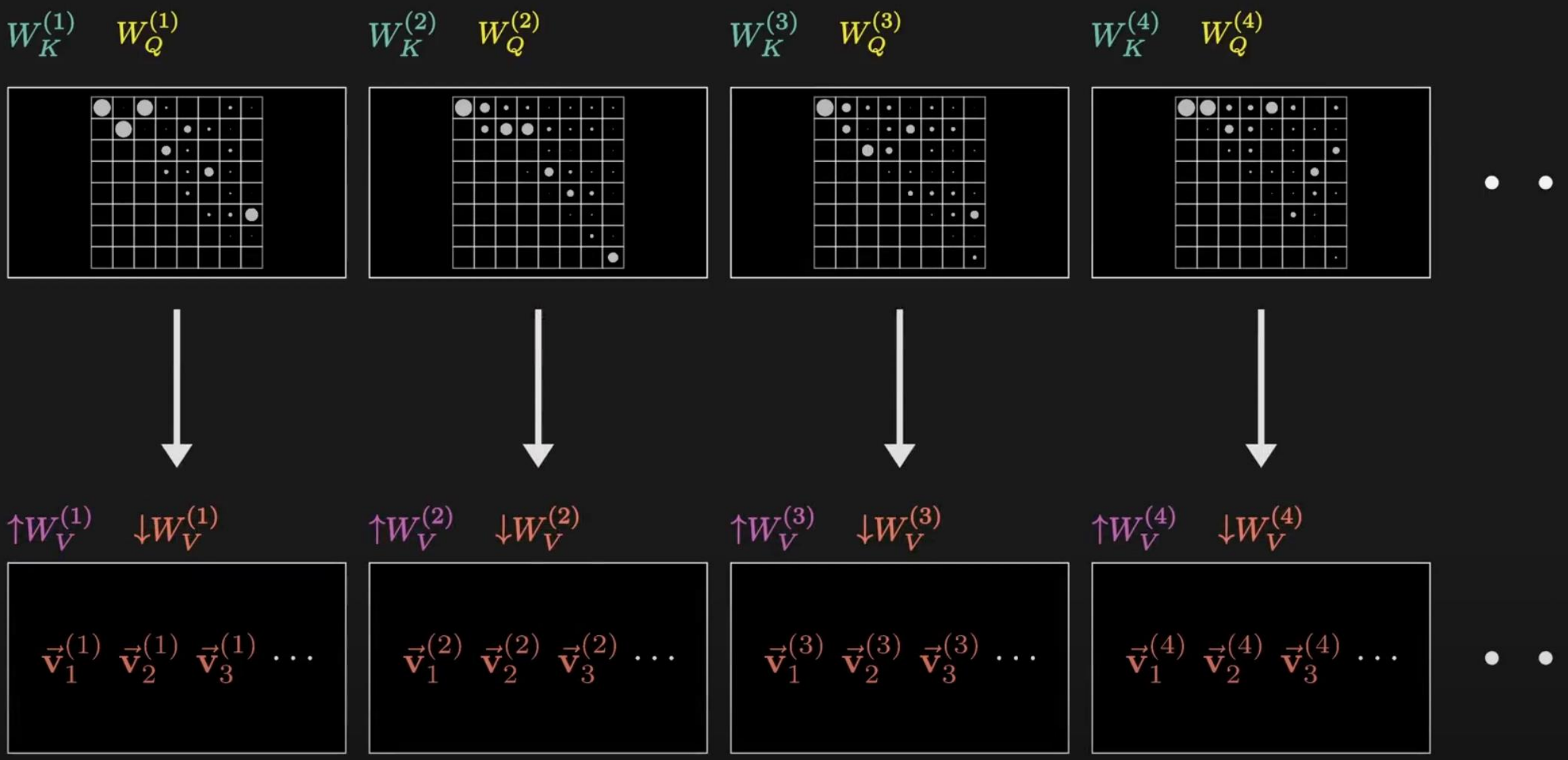
1. It expands the model’s ability to focus on different positions. Yes, in the example above, z1 contains a little bit of every other encoding, but it could be dominated by the the actual word itself. It would be useful if we’re translating a sentence like “The animal didn’t cross the street because it was too tired”, we would want to know which word “it” refers to.
2. It gives the attention layer multiple “representation subspaces”. As we’ll see next, with multi-headed attention we have not only one, but multiple sets of Query/Key/Value weight matrices (the Transformer uses eight attention heads, so we end up with eight sets for each encoder/decoder). Each of these sets is randomly initialized. Then, after training, each set is used to project the input embeddings (or vectors from lower encoders/decoders) into a different representation subspace.



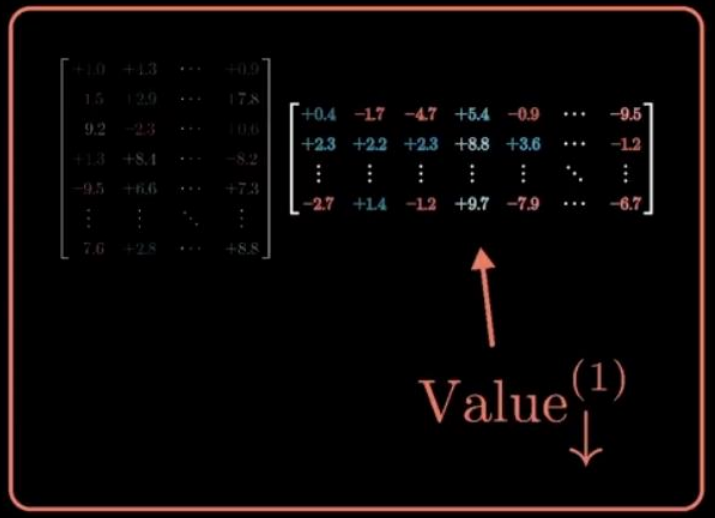
With multi-headed attention, we maintain separate Q/K/V weight matrices for each head resulting in different Q/K/V matrices. As we did before, we multiply X by the WQ/WK/WV matrices to produce Q/K/V matrices.

If we do the same self-attention calculation we outlined above, just eight different times with different weight matrices, we end up with eight different Z matrices

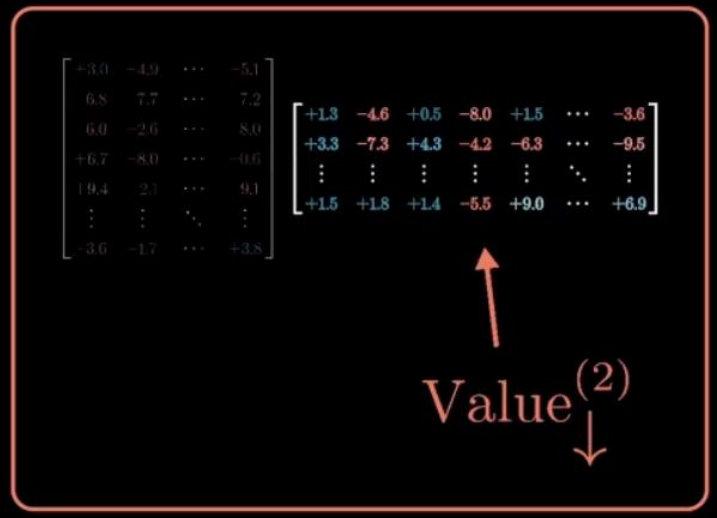



$W_K^{(1)}$ $W_Q^{(1)}$
$W_K^{(2)}$ $W_Q^{(2)}$
$W_K^{(3)}$ $W_Q^{(3)}$
$W_K^{(4)}$ $W_Q^{(4)}$
$\uparrow W_V^{(1)}$ $\downarrow W_V^{(1)}$
$\uparrow W_V^{(2)}$ $\downarrow W_V^{(2)}$
$\uparrow W_V^{(3)}$ $\downarrow W_V^{(3)}$
$\uparrow W_V^{(4)}$ $\downarrow W_V^{(4)}$
$\vec{\mathbf{v}}_1^{(1)}$ $\vec{\mathbf{v}}_2^{(1)}$ $\vec{\mathbf{v}}_3^{(1)}$ $\cdots$
$\vec{\mathbf{v}}_1^{(2)}$ $\vec{\mathbf{v}}_2^{(2)}$ $\vec{\mathbf{v}}_3^{(2)}$ $\cdots$
$\vec{\mathbf{v}}_1^{(3)}$ $\vec{\mathbf{v}}_2^{(3)}$ $\vec{\mathbf{v}}_3^{(3)}$ $\cdots$
$\vec{\mathbf{v}}_1^{(4)}$ $\vec{\mathbf{v}}_2^{(4)}$ $\vec{\mathbf{v}}_3^{(4)}$ $\cdots$

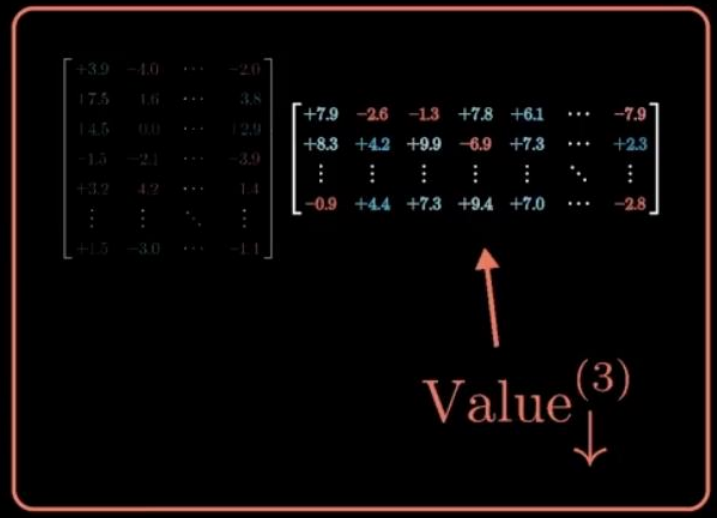
Head 1
Value map



Head 2
Value map



Head 3
Value map



Output matrix

$$\begin{bmatrix} \text{Value}^{(1)} & & & & \text{Value}^{(2)} & & & & \text{Value}^{(3)} & & & & \\ +1.0 & +4.3 & \cdots & +0.9 & +3.0 & -4.9 & \cdots & -5.1 & +3.9 & -4.0 & \cdots & -2.0 & \\ -1.5 & +2.9 & \cdots & +7.8 & -6.8 & -7.7 & \cdots & -7.2 & +7.5 & +1.6 & \cdots & +3.8 & \\ +9.2 & -2.3 & \cdots & +0.6 & -6.0 & -2.6 & \cdots & -8.0 & +4.5 & +0.0 & \cdots & +2.9 & \\ +1.3 & +8.4 & \cdots & -8.2 & +6.7 & -8.0 & \cdots & -0.6 & -1.5 & +2.1 & \cdots & -3.9 & \cdots \\ -9.5 & +6.6 & \cdots & +7.3 & +9.4 & +2.1 & \cdots & -9.1 & +3.2 & -4.2 & \cdots & -1.4 & \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \\ -7.6 & +2.8 & \cdots & +8.8 & -3.6 & -1.7 & \cdots & +3.8 & +1.5 & +3.0 & \cdots & -1.4 & \end{bmatrix}$$

This leaves us with a bit of a challenge. The feed-forward layer is not expecting eight matrices – it's expecting a single matrix (a vector for each word). So we need a way to condense these eight down into a single matrix.

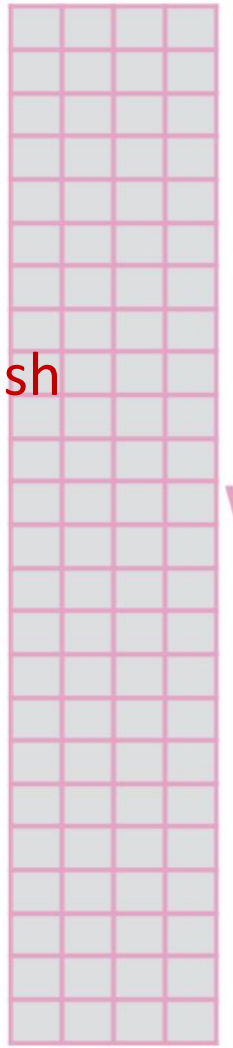
How do we do that? We concat the matrices then multiple them by an additional weights matrix WO.

1) Concatenate all the attention heads



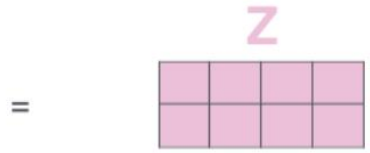
2) Multiply with a weight matrix W^O that was trained jointly with the model

X

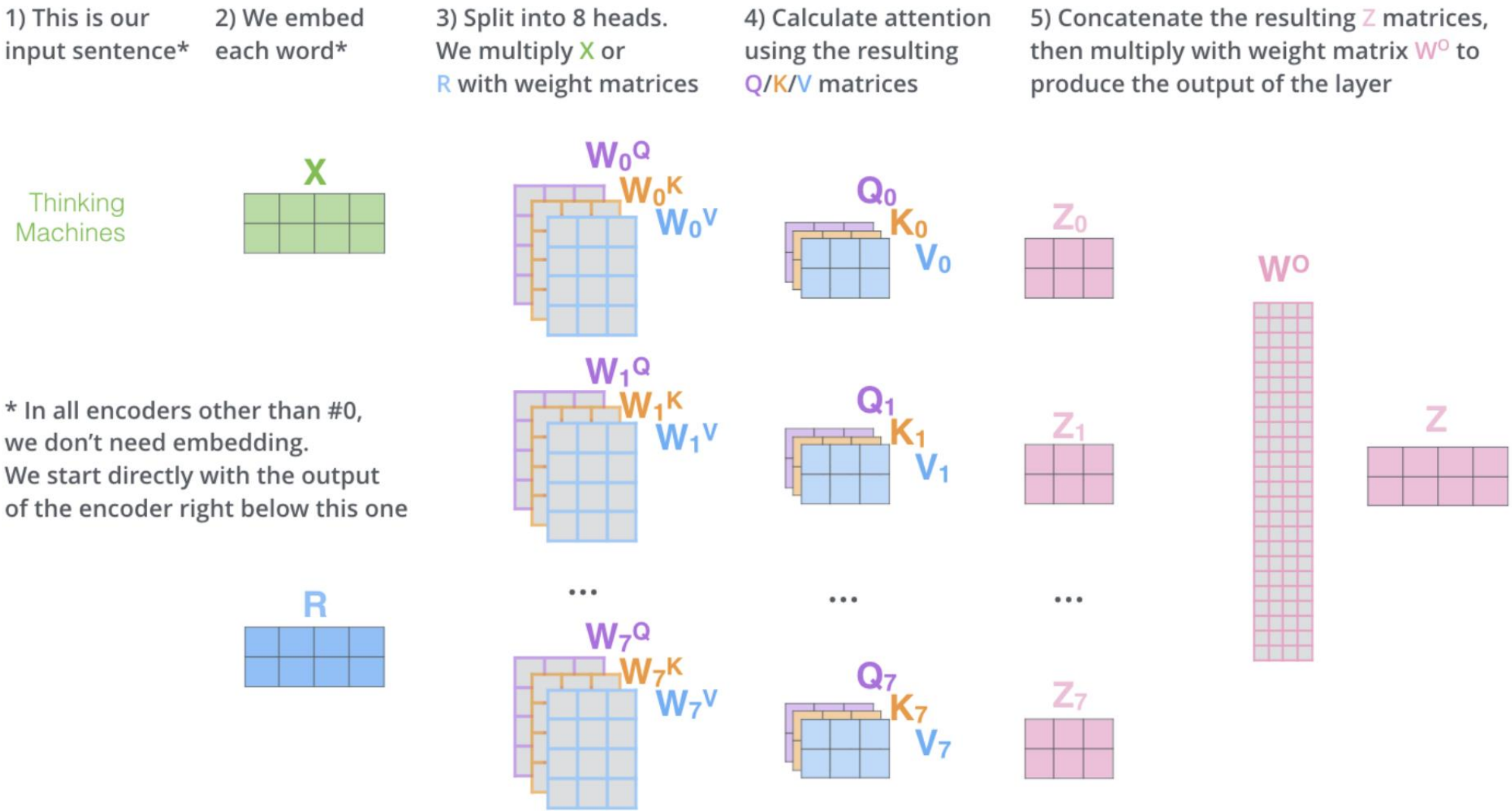


Seems odd we calculate them separately then concatenate and then squish

3) The result would be the Z matrix that captures information from all the attention heads. We can send this forward to the FFNN

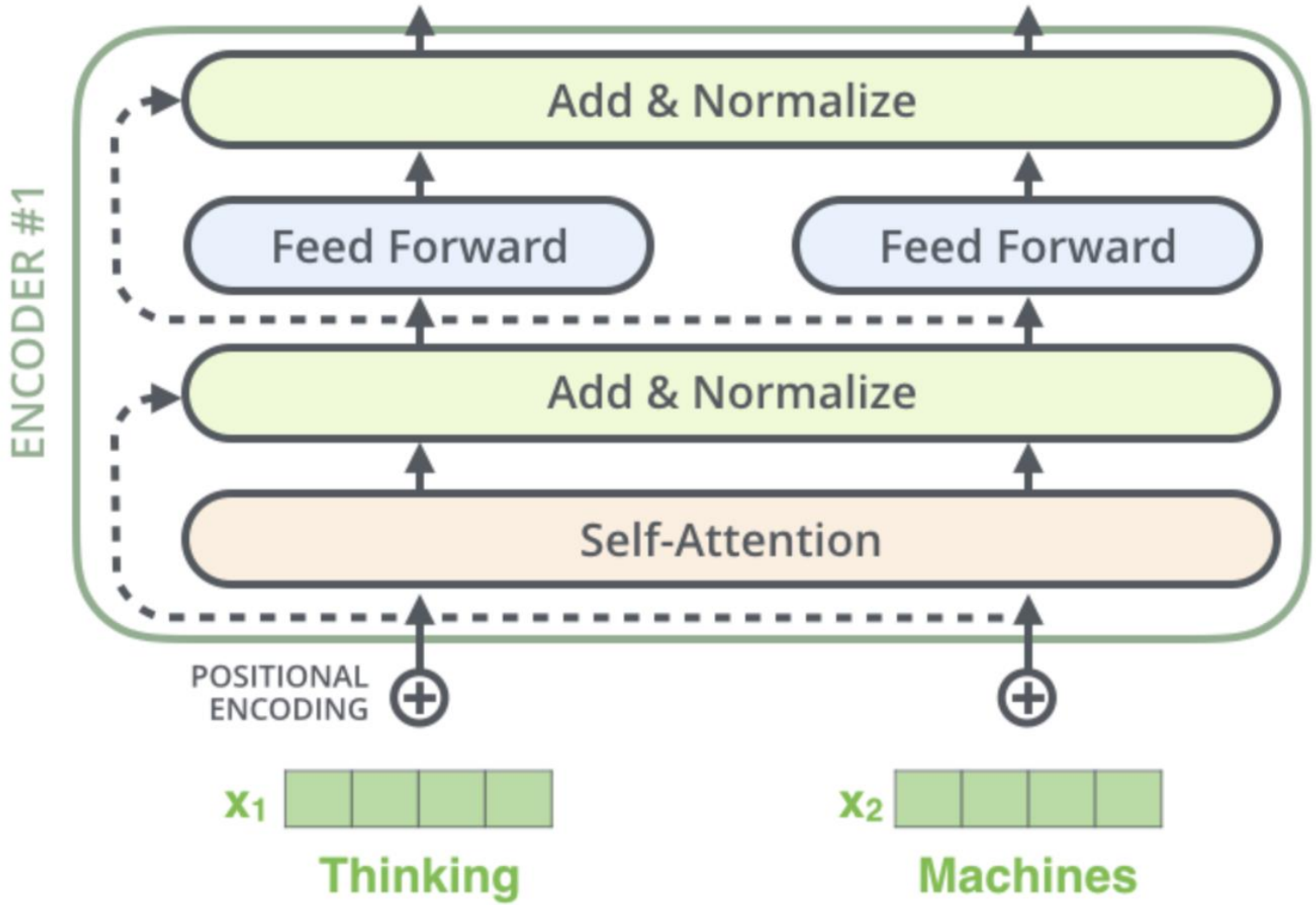


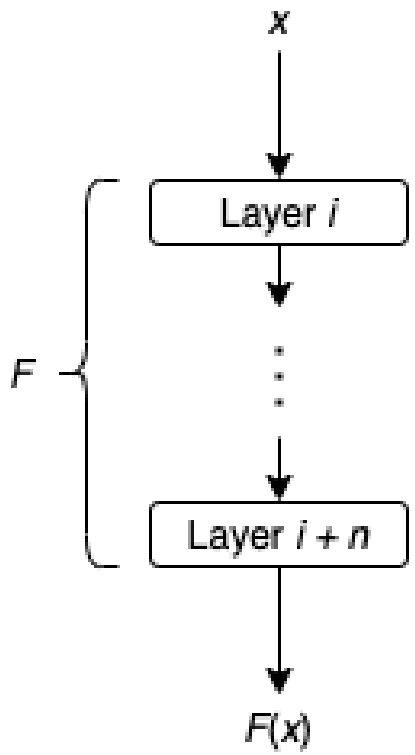
That's pretty much all there is to multi-headed self-attention. It's quite a handful of matrices, I realize. Let me try to put them all in one visual so we can look at them in one place



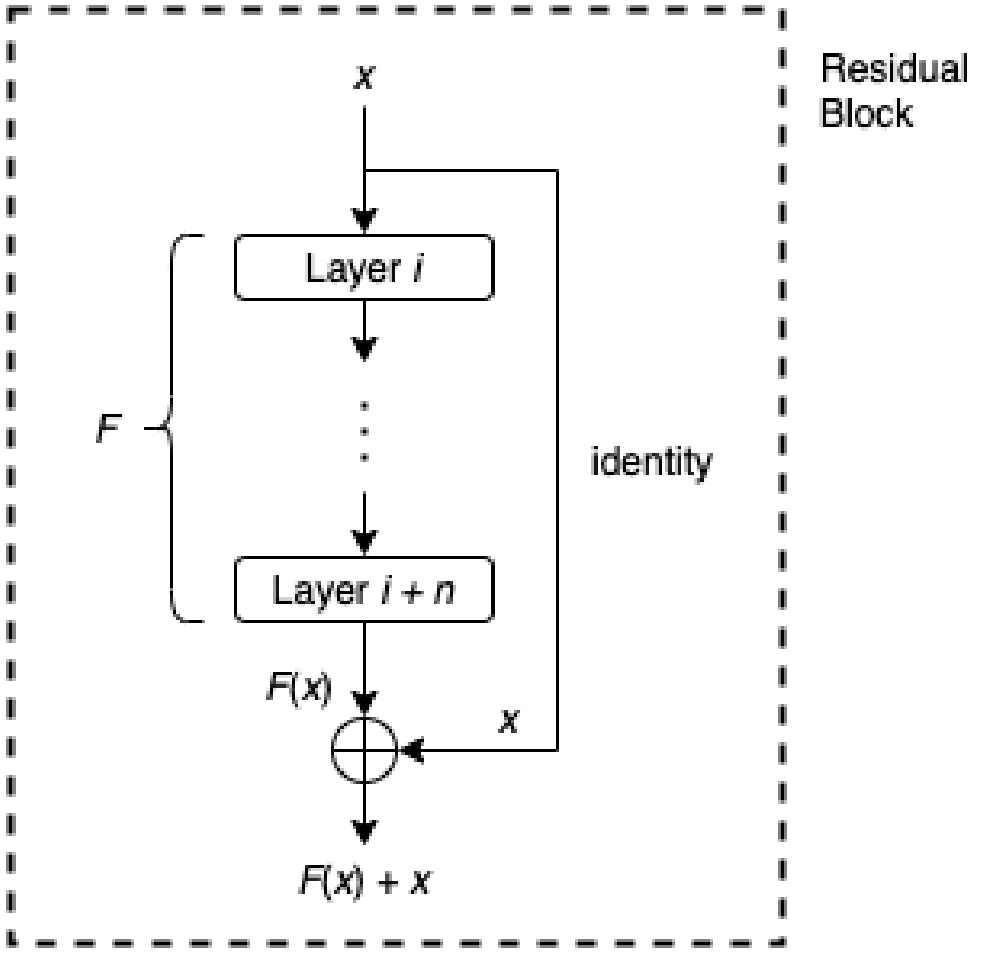
The Residuals

One detail in the architecture of the encoder that we need to mention before moving on, is that each sub-layer (self-attention, ffnn) in each encoder has a residual connection around it, and is followed by a layer-normalization step.

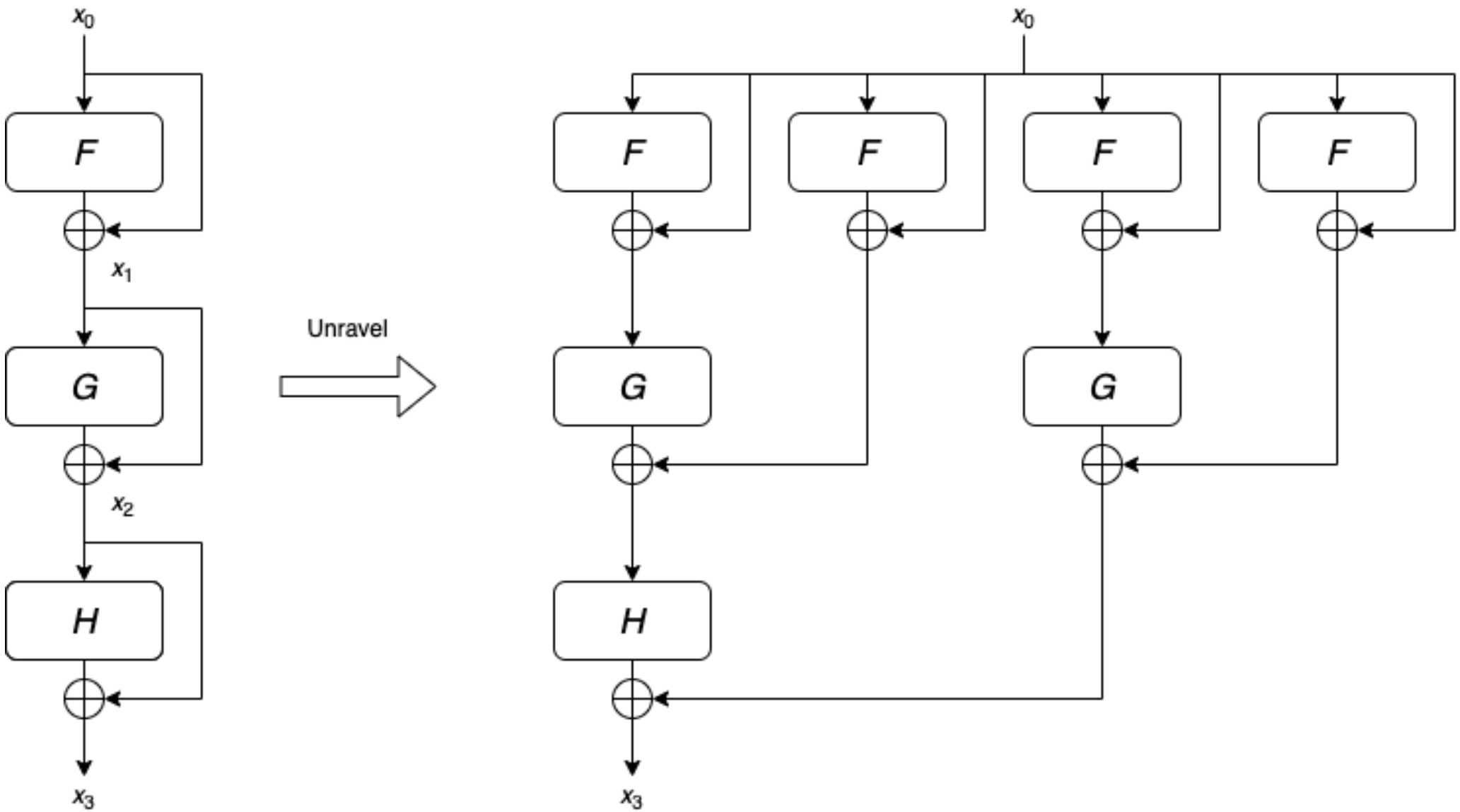




Traditional Feedforward
without Residual Connection



With Residual Connection


x_0
F
x_1
G
x_2
H
x_3
Unravel
x_0
F
F
F
F
G
G
H
x_3

Ways residual connections help

Residual connections, introduced in ResNet (Residual Networks), play a vital role in enabling deep neural networks to **incrementally learn** by addressing problems like **vanishing gradients** and **degraded learning** that often plague very deep mod

1. **Eases Optimization in Deeper Networks:**
• Without residual connections, deep networks need to learn complex transformations all at once.
• Residual connections allow the intermediate layers to focus on learning **incremental changes** to the input. They can treat the output of each block as an enhancement of the input instead of transforming it entirely.
Analogy: Imagine writing a long essay—rather than starting over from scratch every time, you make incremental edits to improve what already exists.

2. **Addressing Vanishing Gradients:**
• In deep networks, gradients can shrink as they propagate backward through layers, leading to poor weight updates.
• Residual connections allow **gradients to flow more freely** through the skip connections, ensuring that learning signals reach the earlier layers effectively.

3. **Facilitates Layer-Wise Refinement:**
• Each residual block can learn to **fine-tune** the output from the previous layers, ensuring that the network gradually improves.
• Even if some blocks learn nothing useful initially, the residual connection ensures that the identity function (input = output) is preserved, so the model can build on already-learned features without degradation.

Ways residual networks help

4. **Encourages Modular Learning:**
• Incremental learning benefits from modularity, where individual components of a system can refine different aspects over time.
• Residual blocks function as modular units that **independently refine or improve features** in isolation without disrupting previously learned representations.

5. **Resilience to Overfitting and Better Generalization:**
• Since residual blocks can "skip" learning unnecessary transformations when the data does not require them, the network is less likely to overfit to noise.
• The model effectively learns the **necessary corrections** to existing features rather than over-complicating its transformations.

6. **Supports Smooth Gradient Flow in Transfer Learning:**
• In incremental learning tasks, such as transfer learning or continual learning, pre-trained networks often need to adapt to new tasks gradually.
• Residual connections make it easier to **adapt existing representations incrementally** because the network doesn't discard previously learned features—it refines them through residual updates.

The Feed Forward Network in Transformers

In the Transformer algorithm, **Feed-Forward Networks (FFNs)** are used within each layer to apply **non-linear transformations** to the data, helping the model learn complex relationships. They are crucial for processing the output from the **self-attention mechanism**.

1. **Transformation and Feature Extraction:**

• The FFN takes the output of the self-attention block and applies a series of **non-linear transformations** to extract richer representations.

• This helps the model capture intricate patterns in the input data beyond what the attention mechanism identifies.

2. **Structure:**

• Each FFN consists of two linear layers with a non-linear activation function (usually **ReLU or GELU**) in between:

$$\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$$

- W_1 and W_2 are learned weight matrices, and b_1, b_2 are biases.

3. **Independence Across Positions:**

• Unlike self-attention, the FFN operates **independently on each position** (token) in the sequence. It applies the same transformation across all tokens in parallel.

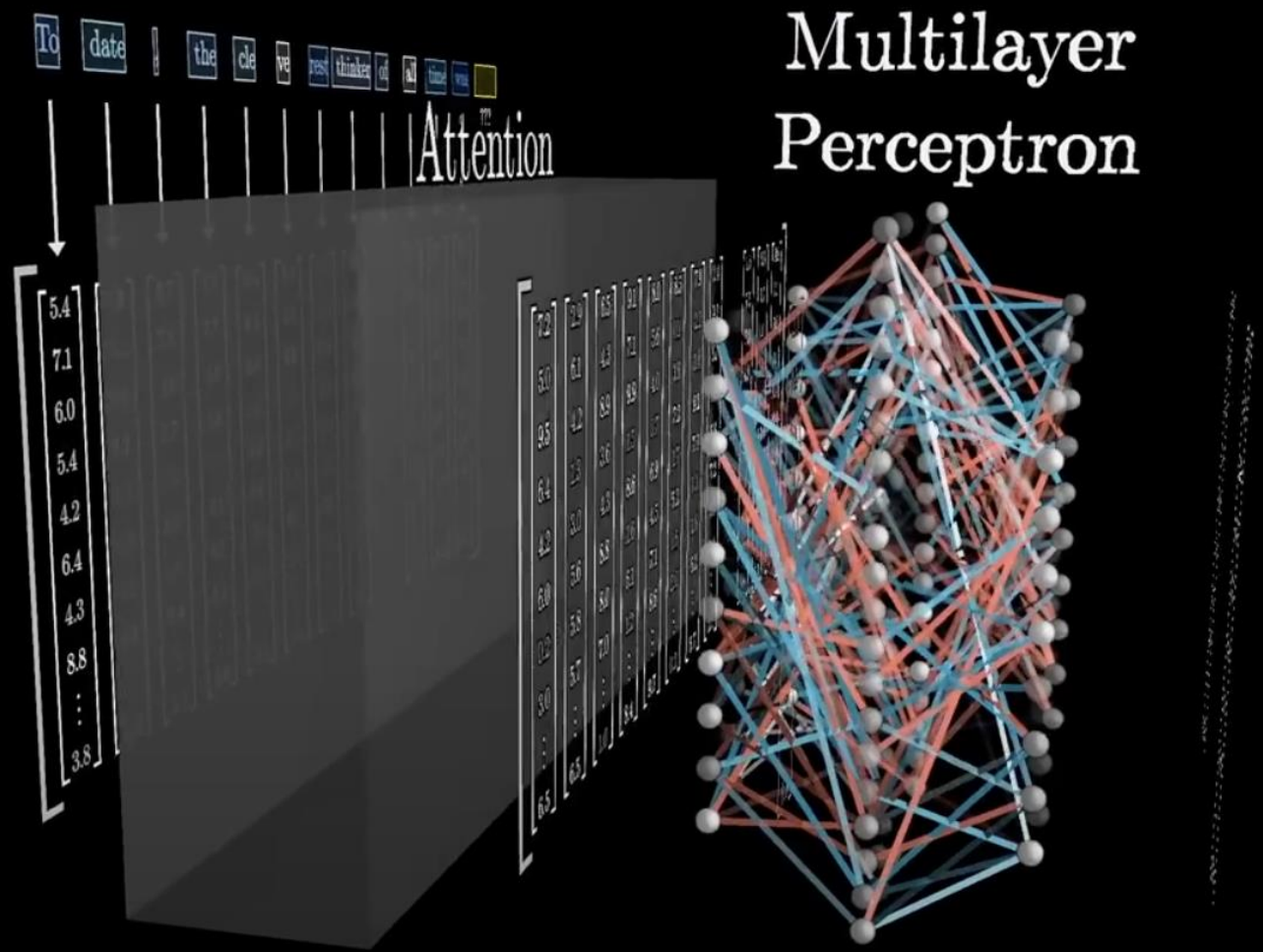
The Feed Forward Networks in Transformers

4. **Adding Depth to Learning:**
• FFNs enable deeper feature extraction within each Transformer layer, complementing the **contextual dependencies** learned by attention.

5. **Contribution to Residual Learning:**
• The FFN output is added to the input of the block via **residual connections**, helping the model **incrementally learn** and prevent gradient issues.

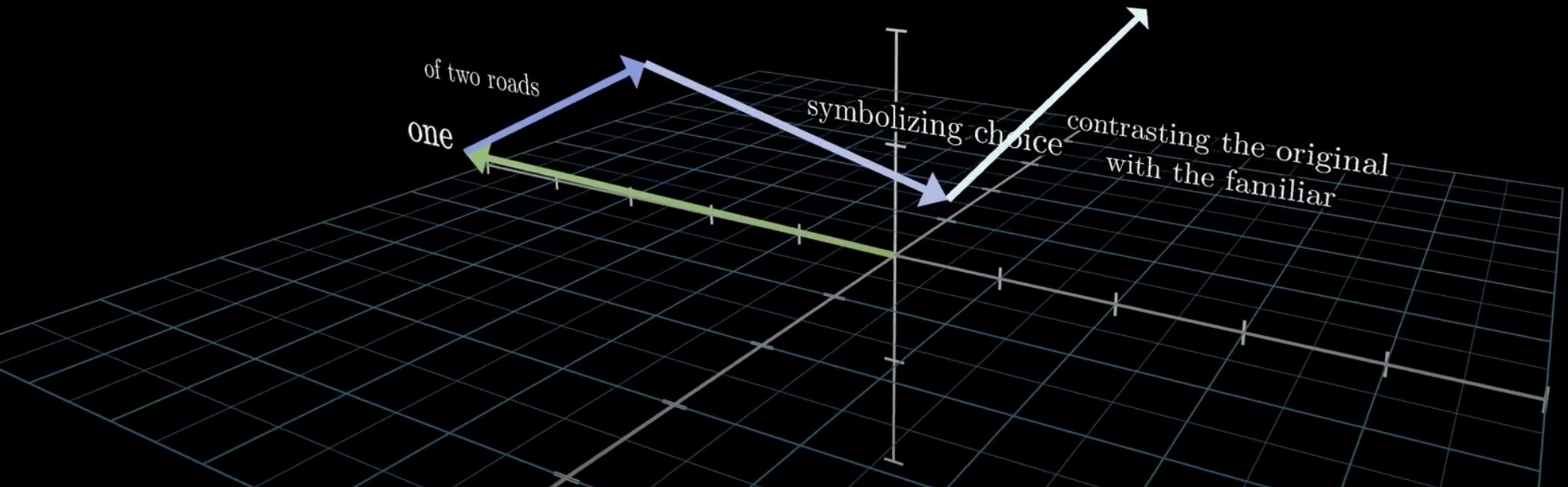
About 1/3 of What

Attention is ~~All~~ You Need

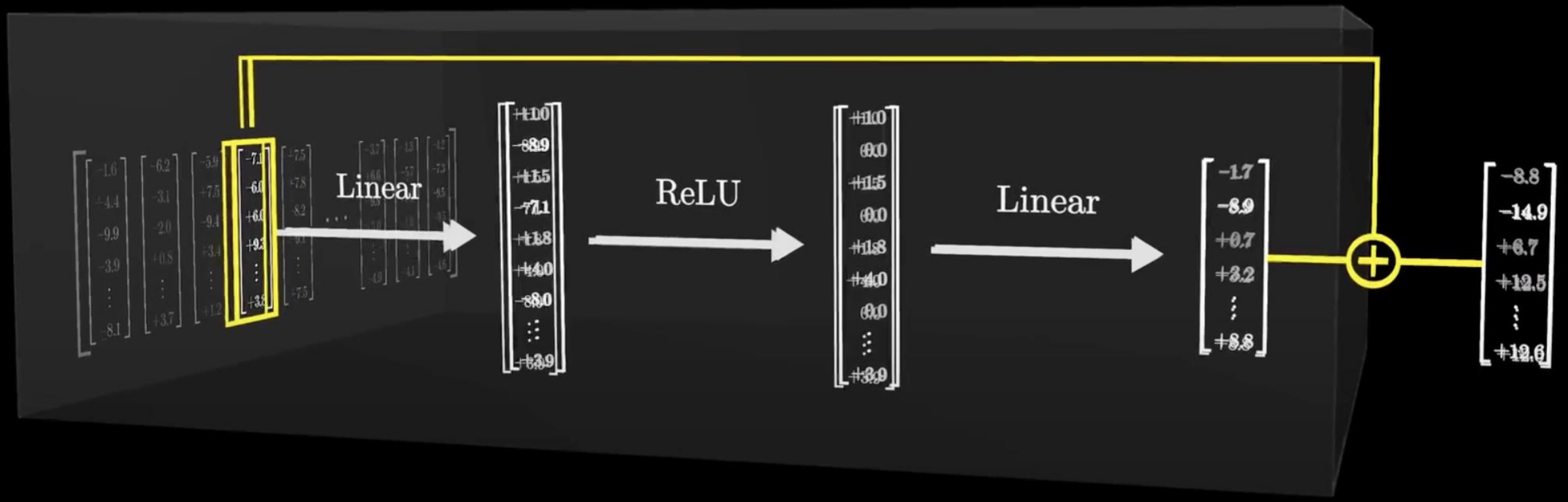

To
date
the
thinker
of
Attention
Multilayer
Perceptron

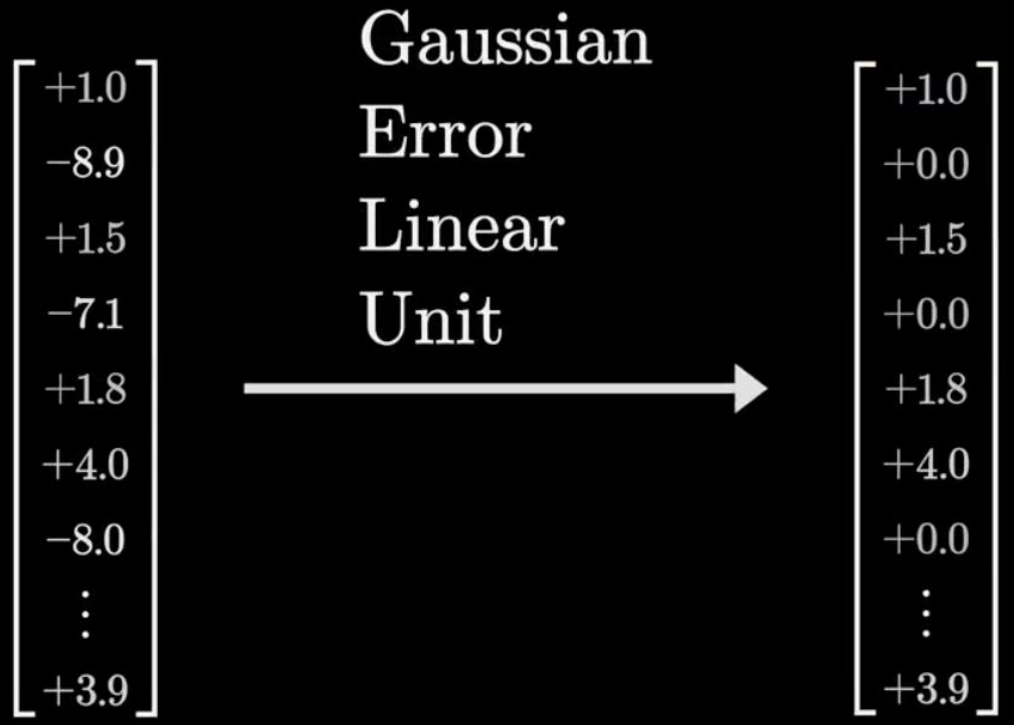
⋮

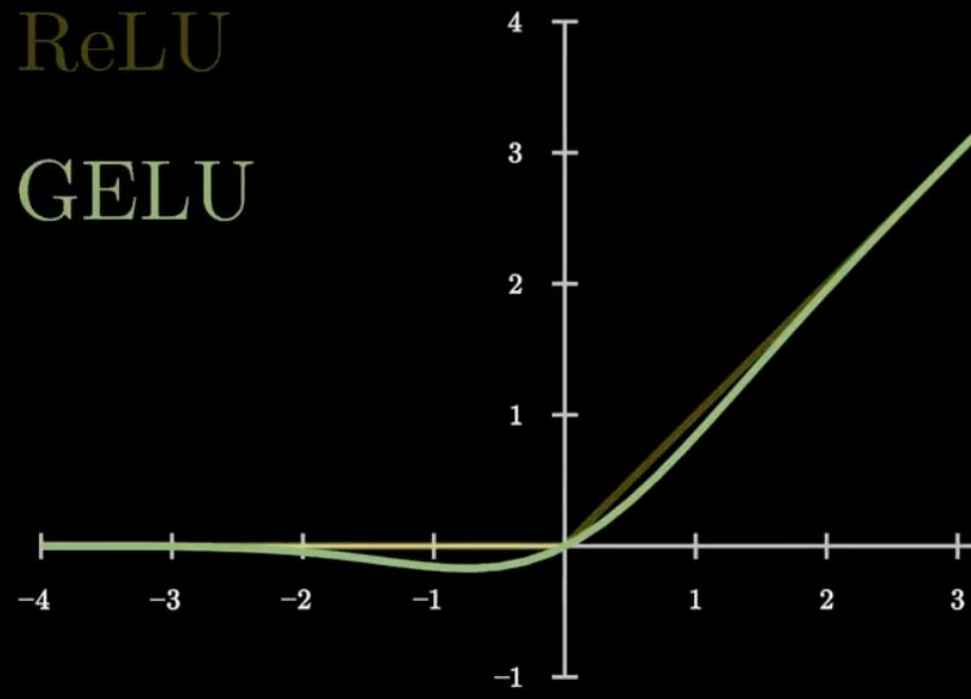
Two roads diverged in a wood, and I—
I took the one less traveled by,

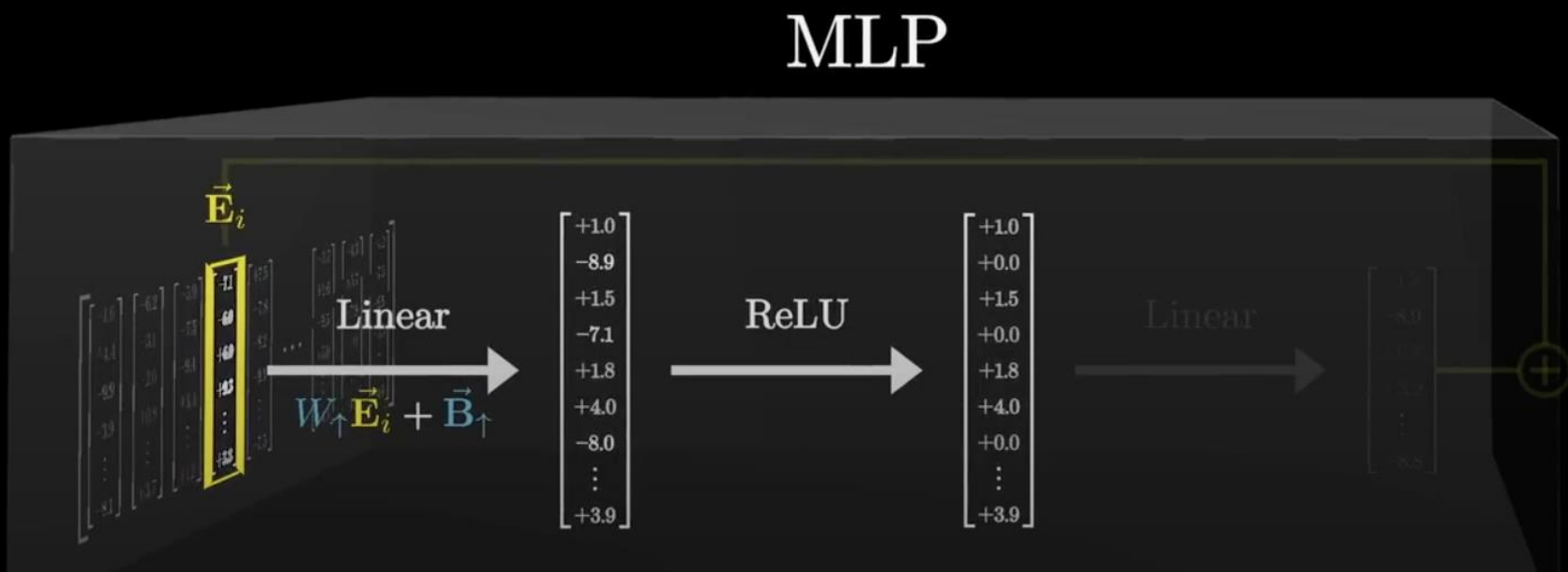


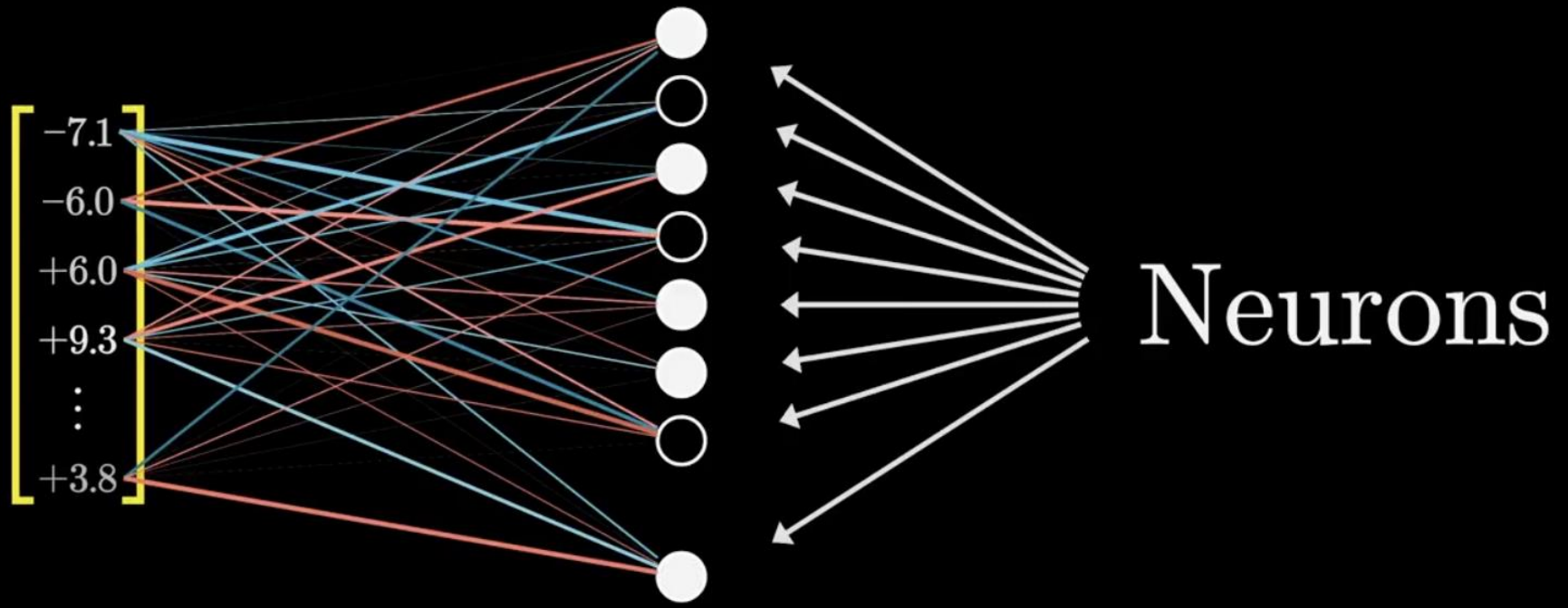
MLP

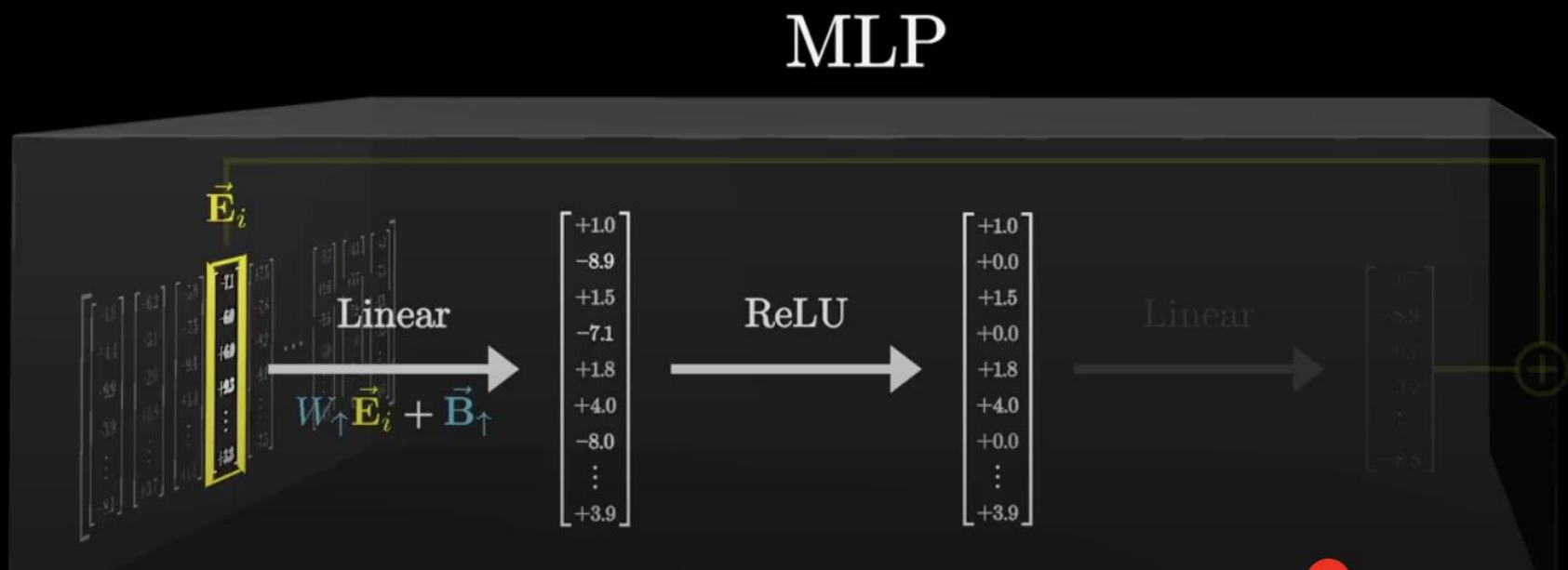


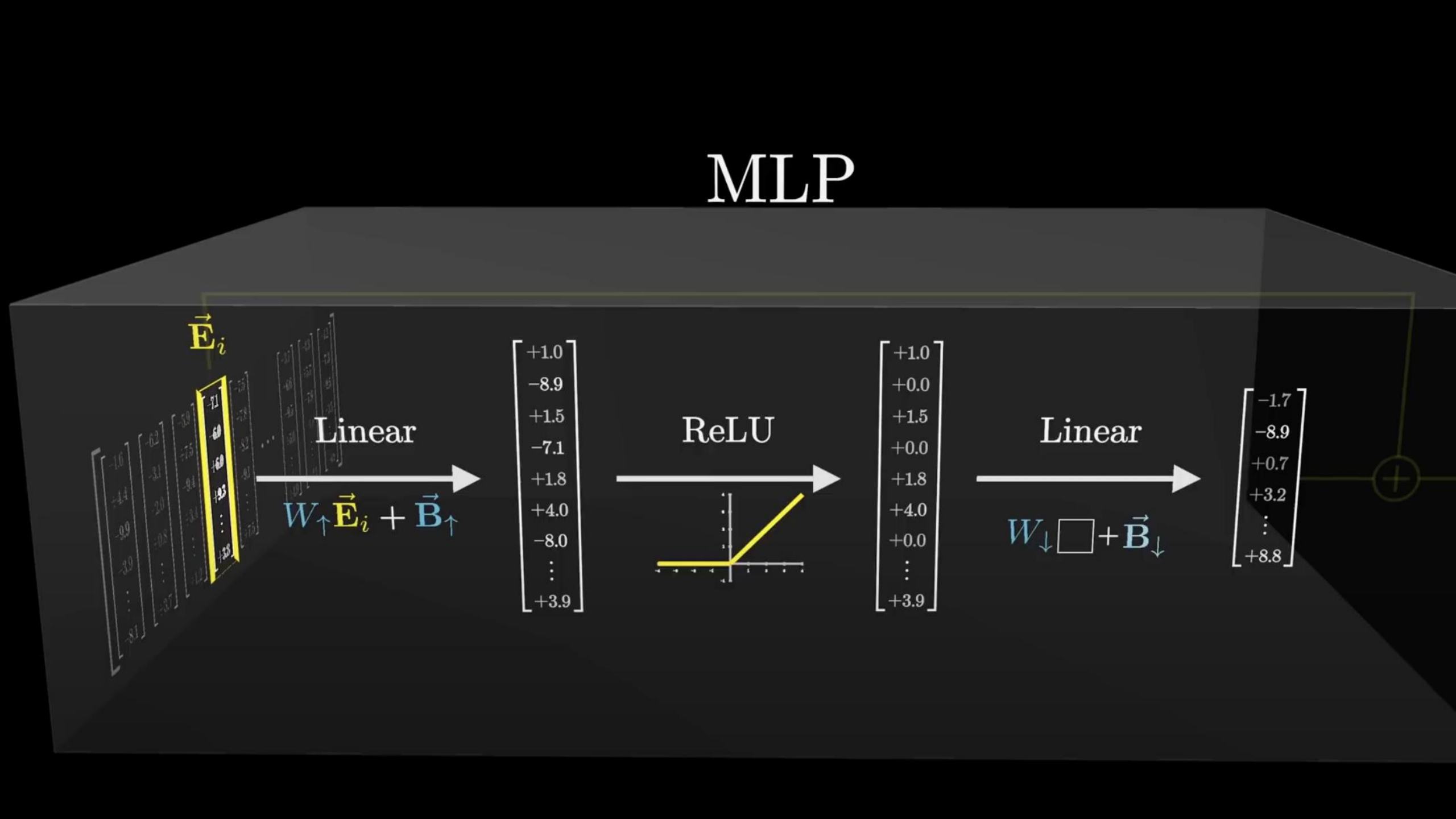

+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
Gaussian
Error
Linear
Unit
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9

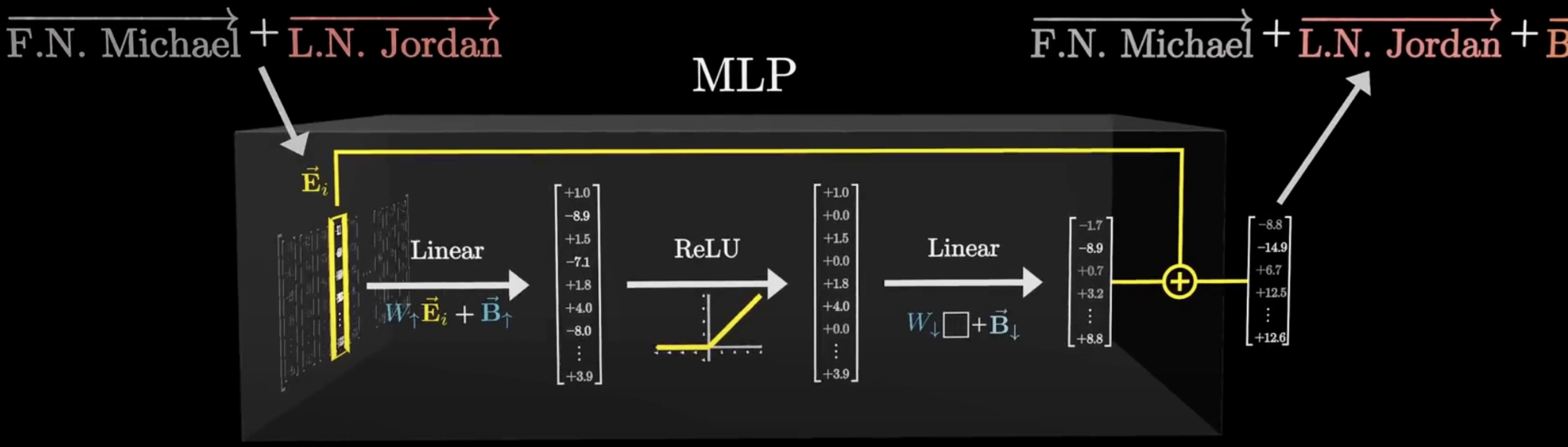

ReLU
GELU
4
3
2
1
−1
−4
−3
−2
−1
1
2
3

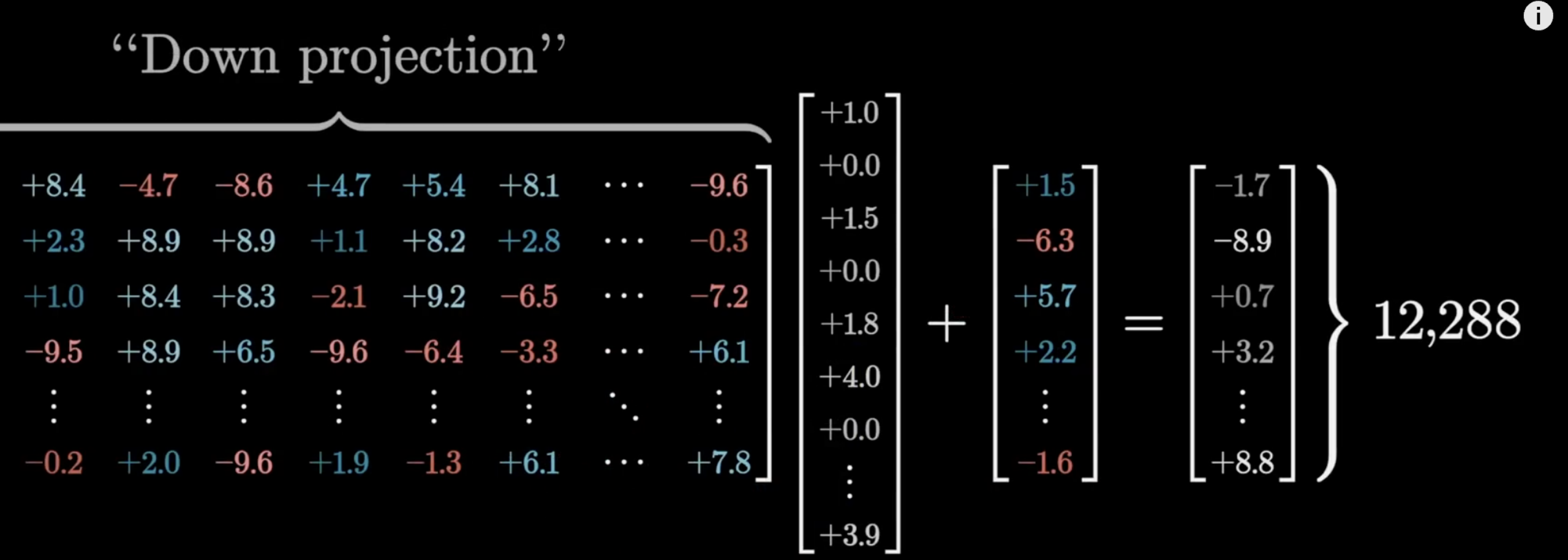

MLP
$\vec{E}_i$
Linear
$W_\uparrow \vec{E}_i + \vec{B}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear


−7.1
−6.0
+6.0
+9.3
+3.8
Neurons


MLP
$\vec{E}_i$
Linear
$W_\uparrow \vec{E}_i + \vec{B}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear


MLP
$\vec{\mathbf{E}}_i$
Linear
$W_\uparrow \vec{\mathbf{E}}_i + \vec{\mathbf{B}}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear
$W_\downarrow \square + \vec{\mathbf{B}}_\downarrow$
−1.7
−8.9
+0.7
+3.2
+8.8


F.N. Michael + L.N. Jordan
MLP
$\vec{E}_i$
Linear
$W_\uparrow \vec{E}_i + \vec{B}_\uparrow$
+1.0
−8.9
+1.5
−7.1
+1.8
+4.0
−8.0
+3.9
ReLU
+1.0
+0.0
+1.5
+0.0
+1.8
+4.0
+0.0
+3.9
Linear
$W_\downarrow \square + \vec{B}_\downarrow$
−1.7
−8.9
+0.7
+3.2
+8.8
−8.8
−14.9
+6.7
+12.5
+12.6
F.N. Michael + L.N. Jordan +



rojection	$\overset{49{,}152}{\text{n_neurons}} * \overset{12{,}288}{\text{d_embed}} = 603{,}979{,}776$ per layer
n-projection	$\overset{12{,}288}{\text{d_embed}} * \overset{49{,}152}{\text{n_neurons}} = 603{,}979{,}776$ per layer
mbedding	$\overset{50{,}257}{\text{n_vocab}} * \overset{12{,}288}{\text{d_embed}} = 617{,}558{,}016$

GPT-3

Total weights:

175,181,291,520

Embedding	$\overset{12,288}{\text{d_embed}} * \overset{50,257}{\text{n_vocab}}$	$= 617,558,016$
Key	$\overset{128}{\text{d_query}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Query	$\overset{128}{\text{d_query}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Value	$\overset{128}{\text{d_value}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Output	$\overset{12,288}{\text{d_embed}} * \overset{128}{\text{d_value}} * \overset{96}{\text{n_heads}} * \overset{96}{\text{n_layers}}$	$= 14,495,514,624$
Up-projection	$\overset{49,152}{\text{n_neurons}} * \overset{12,288}{\text{d_embed}} * \overset{96}{\text{n_layers}}$	$= 57,982,058,496$
Down-projection	$\overset{12,288}{\text{d_embed}} * \overset{49,152}{\text{n_neurons}} * \overset{96}{\text{n_layers}}$	$= 57,982,058,496$
Unembedding	$\overset{50,257}{\text{n_vocab}} * \overset{12,288}{\text{d_embed}}$	$= 617,558,016$

Feed-forward layers constitute two-thirds of a transformer model's parameters, yet their role in the network remains under-explored. We show that feed-forward layers in transformer- based language models operate as key-value memories, where each key correlates with textual patterns in the training examples, and each value induces a distribution over the output vocabulary. Our experiments show that the learned patterns are human-interpretable, and that lower layers tend to capture shallow patterns, while upper layers learn more semantic ones. The values complement the keys' input patterns by inducing output distributions that concentrate probability mass on tokens likely to appear immediately after each pattern, particularly in the upper layers. Finally, we demonstrate that the output of a feed-forward layer is a composition of its memories, which is subsequently refined throughout the model's layers via residual connections to produce the final output distribution.

